

Teaching a Flow-Matching TTS to Read Kanji: Text-Only Self-Distillation of the Text Encoder of Irodori-TTS

Claude (Anthropic)* Takuma Mori

Technical report, October 2026

<https://github.com/takuma104/Irodori-TTS-Experiments>

Abstract

Japanese text-to-speech (TTS) has to choose among several readings of most kanji, such as on’yomi and kun’yomi, from context. We study Irodori-TTS-v4.1-Small, a rectified-flow diffusion transformer (DiT) that reads raw Japanese text through a pretrained Modern-BERT encoder. On the JKYB-Parakeet benchmark, most of its reading errors are systematic. Writing the target word in katakana fixes 85% of them, so the frozen acoustic model can already pronounce the right reading; the text path chooses the wrong one. We turn this into supervision. The frozen model, given the katakana sentence, is the teacher. A trainable copy of the text path (the top 4 BERT layers and the projector), given the kanji sentence, is the student. The student is trained so that the frozen DiT’s velocity and duration predictions match the teacher’s. No speech recordings are needed: target words come from dictionaries, and their sentences from an LLM, Wikipedia and the ruby (reading annotations) of Aozora Bunko. Together with preservation losses, target-window loss weighting, more sentences per word and checkpoint averaging, the released model raises target reading accuracy on held-out Aozora Bunko ruby from 62.5% to 67.5% and on JKYB-Parakeet from 94.0% to 95.7%. General sentences show no measurable regression. We also report where the method stops helping (untrained words, memorization of sentences) and discuss its use for other languages, such as Mandarin polyphonic characters.

1 Introduction

Most kanji have several readings, and the reading depends on the word and the context: 生 is *sei* in 生活, *nama* in 生ビール, *i* in 生きる. Classic Japanese TTS front-ends resolve readings with a morphological analyzer and a pronunciation dictionary [17, 18, 19]. Recent end-to-end models instead read raw text, so the reading is decided implicitly inside the network and only for words that the paired speech data covered.

*Claude (Opus 5.5), working in Claude Code, planned and implemented the experiments, ran them, analyzed the results and wrote this report. Takuma Mori proposed the idea of fine-tuning only the text encoder with text-only data, set the goals (fewer on/kun swaps, no benchmark data in the production model), steered the data choices, did the listening tests and decided the releases.

Irodori-TTS-v4.1-Small [1] is such a model. On JKYB-Parakeet [10, 11], a benchmark of 13,536 sentences each with a tagged target word, it reads 93.9% of the targets correctly. When we listened to its errors, swaps between on’yomi and kun’yomi (音訓の取り違い) were the most noticeable. This report describes how we reduced such errors *without new speech data and without touching the acoustic model*:

- a diagnosis showing that the errors come from the text path (§3);
- kana-substitution self-distillation, which fine-tunes only the text encoder (§4);
- a data pipeline built from dictionaries, an LLM, Wikipedia and Aozora Bunko ruby, which does not use the benchmark (§5);
- findings on loss dilution, memorization versus sentence variety, drift on untrained words and checkpoint averaging (§6).

The checkpoint is released on Hugging Face¹ and works with the original inference code.

2 Irodori-TTS and Echo-TTS

Echo-TTS [2] is a multi-speaker TTS model. A diffusion transformer generates the continuous latents of an audio autoencoder with rectified flow [3, 4], an approach also known as flow matching. Text, as UTF-8 bytes, goes through a bidirectional transformer encoder. A separate encoder reads patched latents of up to two minutes of reference speech. In each decoder block, the keys and values of the text and speaker encoders are concatenated with those of the denoiser, so that one attention covers self-attention and conditioning. Timesteps condition the blocks through a low-rank adaptive layer norm (AdaLN). Text and speaker conditions are dropped independently during training, which enables classifier-free guidance (CFG) [5] with separate text and speaker scales.

Irodori-TTS [1] follows this design closely. Table 1 lists the components of v4.1-Small:

- **Target:** 32-dimensional latents of a DACVAE codec [6, 7] at 25 Hz, decoded to 48 kHz audio.
- **DiT:** the RF-DiT (12 blocks, width 1280) uses joint attention, low-rank AdaLN, half-RoPE and SwiGLU.

¹<https://huggingface.co/takuma104/Irodori-TTS-v4.1-Small-Yomi>

Table 1: Components of Irodori-TTS-v4.1-Small and what this work trains.

Component	Params	This work
RF-DiT (12 blocks, width 1280)	366M	frozen
ModernBERT-ja (25 layers, 768)	315M	top 4 trained*
Text projector, <code>text_norm</code>	1.7M	trained
Caption projector, norm	1.7M	refit (§4.5)
Speaker encoder	61M	frozen
Duration predictor	22M	frozen

*Plus the final norm; 39.5M trainable parameters in total.

- **Text:** instead of a byte encoder, v4 uses a pretrained Japanese ModernBERT [8, 9], fine-tuned with the model. Its states go through a residual-MLP projector and an RMSNorm (`text_norm`) into a 512-dimensional text condition.
- **Caption:** the same BERT encodes the voice-design caption, through a separate projector.
- **Duration:** a duration predictor (retrained separately in v4.1) estimates the output length from the text state, the speaker and the caption.

Inference uses 40 sampling steps with independent CFG (text 3.0, speaker 5.0). Because the DiT only sees text states, *the choice of reading is made by the text path*.

3 Diagnosis: the text path picks the reading

Evaluation. Each generated sentence is transcribed into kana with kana-whisper [12, 13]. `jkyb-eval` [11] then aligns the transcript with the reference reading and scores the target word as correct or not (target reading accuracy). All numbers use one reference voice (`jvs001` of JVS [15]) and seed 0. The DiT runs in bf16, and the text encoder and codec in fp32. Running everything in bf16 lost 0.4pt; with this mixed setting the base model scores 94.04% (93.91% in fp32). We compare models item by item, count fixed and broken items, and use the exact McNemar test [16].

Kana oracle. We took the 882 errors and 882 correct items of the base model and generated them with three seeds. 75% of the errors were *systematic*: wrong with every seed. We then replaced the target word by its reading in katakana. This fixed 85% of the systematic errors and 95% of the systematic on/kun swaps. It broke 2.2% of the correct items, compared with 1.4–1.5% from changing the seed alone. So the DiT can pronounce the reading; the mistake is the reading the text path chooses. That makes the katakana input a natural teacher.

4 Method

4.1 Kana-substitution self-distillation

For a sentence s with target word w and reading r , let $\tilde{s}$ be s with w replaced by r in katakana. The base model generates a latent x_0 from $\tilde{s}$; the reference speaker is chosen per sentence from 99 JVS speakers or left empty. During training, $x_t = (1 - t)x_0 + t\epsilon$ with t drawn from

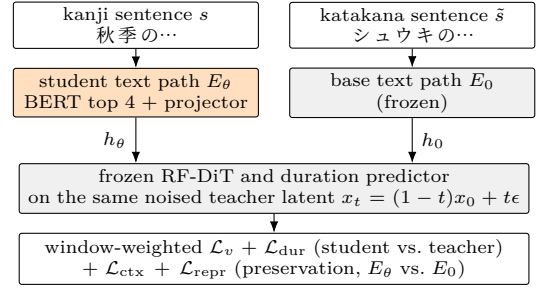


Figure 1: Kana-substitution self-distillation. The teacher is the frozen base model given the target word in katakana; the student text path sees the kanji sentence. Both condition the same frozen DiT on the same noised teacher latent x_t .

a stratified logit-normal distribution. Let $v(\cdot; h)$ be the velocity of the frozen DiT conditioned on text states h and the speaker. The teacher’s states are $h_0 = E_0(\tilde{s})$ from the frozen text path, and the student’s are $h_\theta = E_\theta(s)$. The student is a copy of the top 4 of the 25 BERT layers, the final norm, the text projector and `text_norm`; it shares the lower 21 layers with the frozen model. Per sentence,

$$\mathcal{L}_v = \frac{\sum_j \omega_j \|v_j(x_t; h_\theta) - v_j(x_t; h_0)\|^2}{D \sum_j \omega_j}, \quad (1)$$

$$\mathcal{L}_{\text{dur}} = \text{SmoothL1}_{\beta=0.1}(d(h_\theta), d(h_0)), \quad (2)$$

where j runs over latent frames, $D = 32$, and d is the predicted log length. We use conditional velocities only. The speaker condition is dropped for 20% of the samples, and the caption is empty.

Two preservation losses keep the encoder close to the original wherever the reading should not change:

- $\mathcal{L}_{\text{ctx}}$ is the MSE between $E_\theta(s)$ and $E_0(s)$ over the tokens outside the kanji run that contains w .
- $\mathcal{L}_{\text{repr}}$ is the same MSE on a batch of 64 general Wikipedia sentences.

The total loss is the mean of $\mathcal{L}_v + \mathcal{L}_{\text{dur}}$ over the batch, plus $\mathcal{L}_{\text{ctx}} + \mathcal{L}_{\text{repr}}$ (weights 1). The text path runs in fp32 and the frozen DiT in bf16. We use AdamW with learning rates $3 \cdot 10^{-4}$ (projector) and 10^{-4} (BERT), 200 warm-up steps, cosine decay to 10%, batch size 32 and gradient clipping at 1. On one RTX 5090 this runs at about 0.86 steps/s.

4.2 Hard mining

The base model first generates *every* training sentence from the kanji text, and the result is scored with ASR. Only misread sentences get the kana teacher, and only if the teacher’s own audio is read correctly (86–97% are). Sentences that the base model already reads correctly keep its own latent and the kanji text as the teacher. They ask the student to change nothing. Kana-teacher rows are repeated $3 \times$ per epoch.

4.3 Target-window weighting

In pilot sentences, 87–95% of the teacher–base velocity difference lay in the frames of the target word, which were 23% of all frames. Averaging over all frames therefore dilutes the signal in long sentences. From v2 on,

we weight the frames of the target by $\omega_j = 5$ and the others by 1. The target’s span in the reading is mapped proportionally onto the frames, with a 10% margin on each side.

4.4 Contrast rows and checkpoint averaging

Contrast rows are sentences in which a kanji appears with a reading other than the trained one. The base model reads them correctly, and its own output is kept as the teacher, with the kanji window weighted. They protect the readings of compounds (§5).

Checkpoint averaging takes two students on the same fine-tuning path, θ_A and θ_B , and interpolates them as $(1 - \alpha)\theta_A + \alpha\theta_B$, as in WiSE-FT [24] and model soups [25].

4.5 Export and the shared caption encoder

To release a stock checkpoint, the student’s tensors replace the corresponding tensors of the base model. Because BERT is shared, this also moves the caption states, by 21–24% relative L2 per token. We therefore refit the caption projector and its norm by state matching: on 5.6k LLM-written captions and Wikipedia sentences, the new caption states are fitted to the base model’s (3,000 steps, no audio). This brings the shift down to 6.5–7.6%. For scale, the mean-pooled states of two different captions differ by 40%. In caption-only synthesis, the median pitch then differs from the base model by 0.36–0.45 semitones, against 0.96 without the refit. In fp32, the exported text path is identical to the student.

5 Data generation

Lexicon. JMdict [20] and JmdictFurigana [21] give 211k (word, reading) pairs with the reading of each kanji. We classify each kanji reading against the Joyo table (on, kun, jukujikun, appendix) and assign words to buckets: on’yomi in compounds that the tokenizer splits (A) or keeps as one token (B), kun’yomi (C, D), homographs (E) and jukujikun (F). Contrast words (G) and Aozora ruby words (R) form two more buckets. Table 2 lists the sentence sources:

- **LLM.** Qwen3.6-27B [22], served with vLLM [23], writes 4–8 sentences per word that use the word with the given reading. For homographs, a second multiple-choice call checks the reading.
- **Wikipedia.** An Aho-Corasick search looks for 55k lexicon stems in 3.0M sentences. A sentence is kept only when MeCab+UniDic [17, 18] and Sudachi [19] both segment the stem as one unit and read it as the dictionary does. On 376k spans, the two analyzers agree on 97.1% (99.0% for words with one reading, 94.0% for homographs). Agreement is not correctness, so homograph sentences are also checked by the LLM: 6,758 of 10,528 pass.
- **Aozora Bunko.** The works are split by a hash of the work ID into evaluation (10%) and training works.

Table 2: Training data. “Base err.” is the share of target sentences the base model misreads; “Kana rows” are those that carry the kana teacher.

Set	Source	Sent.	Base err.	Kana rows
Pilot	LLM, 2.6k words	9.3k	29%	1.9k
M3	LLM, 21k words	79k	33%	22.5k
Kun	LLM, Joyo kun	31k	9%	2.1k
Wikipedia	corpus, 12k words	40.6k	14%	4.8k
Aozora	ruby, 1k works	50.3k	37%	16.4k
Variety	LLM, 8.9k words	48.7k	79%	36.3k
Aozora+	ruby, +2k works	67.0k	39%	23.0k
Contrast	Wikipedia	13.3k	–	–
General	JVS text, Wiki	4.6k+200k	–	–

Ruby readings that match a dictionary or analyzer reading become targets; this drops authors’ idiosyncratic readings.

- **Variety.** For every corpus training word that the base model misreads (10k words), the LLM writes 6 new sentences (§6).
- **Contrast.** For each kanji learned as a single-kanji word (秋《とき》), we take common compounds in which it reads differently (秋季) from Wikipedia, up to 4 compounds with 4 sentences each.

Independence from the benchmark. No JKYB sentence was ever used for training. The early stages (pilot to kun, up to S8) took their Joyo reading table from the JKYB keys and held out half of the JKYB target words (group B). From S9 on, the production stages use KANJIDIC2 for the table, exclude nothing, and were judged only on non-JKYB sets.

6 Experiments

Evaluation sets. None of these sets is used for training:

- Aozora ruby: 3,000 items from 385 evaluation works.
- Wikipedia dev: 6,403 sentences of held-out words.
- Pilot dev: 898 LLM sentences of held-out hard words.
- JSUT [14] BASIC5000: sentence kana CER of 2,236 sentences, against the human kana labels.
- JVS: Whisper CER of 500 general sentences.
- JKYB-Parakeet: an external score, partly in-domain for the early stages.

Pilot ablations (Table 3). Training only the projector, or the top 4 layers with a low learning rate, barely changed readings. With a higher learning rate, the top 4 layers learned the training words, and these words were also fixed in unseen JKYB sentences (85.1%→91.5%). But untrained words regressed: JKYB group B fell by 0.89 pt. The preservation losses removed this regression (S4). Overfitting 32 rows took the trained rows from 3.1% to 93.8%, which confirms that the frozen DiT can switch readings when only the text path is trained.

Scaling and stages. We scaled up in stages, each continuing the previous student:

- S5–S7: about 24k LLM words, 33k steps in total.
- S8: one stage of kun’yomi words.
- S9–S10: Wikipedia and Aozora data (v1, 90k steps in total).
- S11–S12: target-window weighting (v2, 150k steps).

Table 3: Pilot ablations (1,500–4,000 steps from the base model). “Train”: 1,000 sentences of training words; dev: held-out words.

Run	Train	Dev	JKYB
Base	71.2	80.7	94.04
S1 projector only	71.0	79.7	–
S2 top 4, lr $10^{-4}/3\cdot 10^{-5}$	72.3	79.3	–
S3 top 4, lr $3\cdot 10^{-4}/10^{-4}$	88.6	79.3	93.48
S4 S3 + $\mathcal{L}_{ctx}, \mathcal{L}_{repr}$	80.1	81.0	94.13

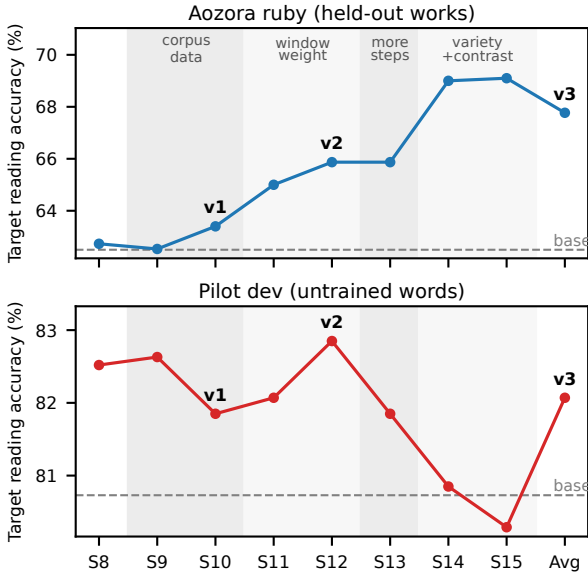


Figure 2: Held-out accuracy by stage. Aozora improves with window weighting and with sentence variety. Pilot dev (untrained words) drifts down with long training. Averaging S12 and S15 (“Avg”, released as v3) keeps most of the Aozora gain and restores pilot dev.

- S13: 60k more steps on the same data.
- S14–S15: variety and contrast data.

Figure 2 shows the two most informative held-out sets, and Table 6 the released checkpoints.

Loss dilution (Table 4). After v1, the student fit its hard Wikipedia and Aozora training rows far worse than the LLM rows (30%/18% against 66%). To find out why, we trained from the base model on hard rows only, with the same number of passes (24 per row). Wikipedia rows fit as fast as the LLM rows; the gap came mostly from fewer passes and from long sentences diluting the target. Weighting the target window helped most. Training 8 layers instead of 4 also helped, at $1.5\times$ the cost, and weaker preservation helped least.

Memorization versus variety. More steps alone (S13) raised the hard Aozora training rows from 42% to 57%. Yet held-out items whose words were trained stayed at 18%: the student memorized *sentences*, not word readings. More sentences per word fixed this. For words that got 6 LLM sentences each (S14), base-misread held-out items were read correctly 34% of the time, against 20% before. Adding 2,000 Aozora works raised newly covered words from 8% to 21%. Words absent from all training data stayed flat at 55%.

Table 4: Fit diagnosis from the base model, with the same number of passes over each set (accuracy on the training rows, %).

Setting	Wikipedia	Aozora	LLM (M3)
Base	12.2	7.6	8.0
Recipe of v1	34.2	18.9	35.1
+ window weight 5	48.2	28.2	–
+ top 8 layers	45.0	26.6	–
+ preservation $\times 0.3$	39.2	21.7	–

Table 5: Interpolating S12 ($\alpha=0$) and S15 ($\alpha=1$).

α	0	0.5	0.7	1
Aozora ruby	65.87	67.77	68.30	69.10
Pilot dev	82.85	82.07	81.63	80.29
Wikipedia dev	85.09	84.96	84.99	85.12

Drift and averaging. Pilot dev (untrained hard words) fell from 82.9% (S12) to 80.3% (S15; 11 fixed, 34 broken, $p = 8\cdot 10^{-4}$). The broken items were mostly untrained compounds read in odd ways (前庭マエニワ → ゼンニワ, 活火山 → イカザン). Common compounds of the contrast kanji held out from training stayed at 97% in every model, so this is a slow drift on untrained words, not a leak of single-kanji readings. Interpolating S12 and S15 trades the two effects smoothly (Table 5); $\alpha = 0.5$ became v3.

General sentences and robustness. JSUT kana CER improves in every release (1.881%→1.713%; 161 sentences better, 99 worse). The JVS Whisper CER moves within noise (13 better, 18 worse), and most of those differences are orthographic variants of the transcript (時/とき). On JKYB, both halves improve: +1.91 pt for the words allowed in early training and +1.41 pt for the held-out half.

Seed check. All other evaluations use seed 0. On the Aozora set with seeds 1 and 2, v3 beats the base model by +5.73 pt with both seeds: 62.27%→68.00% (208 fixed, 36 broken) and 62.13%→67.87% (209/37). With seed 0 the gain is +5.03 pt.

7 Discussion

What is learned. The student learns readings of the words it is trained on, and it applies them in new sentences once each word has varied contexts. It does not learn general reading rules: words absent from training do not improve. Coverage therefore comes from data, and the cheapest data are sentences generated or retrieved for words that the base model misreads.

Limitations. The evaluation is based on ASR, with one reference voice and one seed for most sets (see the seed check above), and kana-whisper or the reference readings can be wrong. Listening was informal. LLM sentences sometimes misuse rare words semantically, even when the reading is right. The shared BERT couples the text and caption paths, and emoji tokens shift more (12–14%) than other tokens. Long continued training drifts on untrained words; averaging limits this

Table 6: Released checkpoints, evaluated as exported files. Accuracy (%) up, CER (%) down. v3 vs. base (fixed/broken): Aozora 200/49 ($p=8 \cdot 10^{-23}$), Wikipedia 126/66 ($p=2 \cdot 10^{-5}$), pilot 31/20 ($p=0.16$), JKYB 320/96 ($p=3 \cdot 10^{-29}$).

	Base	v1	v2	v3
Aozora ruby	62.50	63.30	65.80	67.53
Wikipedia dev	84.04	84.37	85.12	84.98
Pilot dev	80.73	–	82.74	81.96
JSUT kana CER	1.881	1.799	1.778	1.713
JSUT Whisper CER	5.625	5.650	5.683	5.717
JKYB-Parakeet	94.04	95.06	95.46	95.69
on / kun	93.0/95.9	94.5/96.3	95.0/96.5	95.2/96.8

but does not remove it.

Other languages. The recipe needs two things. First, a TTS model whose text encoder is separate from a frozen acoustic model. Second, an alternative spelling that the frozen model reads unambiguously. In Japanese, katakana plays this role. Mandarin polyphonic characters (多音字, e.g. 行 *xíng/háng*, 长 *cháng/zhǎng*) are the analogous problem for character-input TTS. There, the teacher input could replace the character with an unambiguous homophone or with pinyin, if the model reads pinyin. Training sentences can come from polyphone dictionaries and corpora [26]. We have not tested this.

8 Conclusion

Fine-tuning only the top of the text encoder against a kana-reading teacher fixes kanji readings of a frozen flow-matching TTS without any new speech. The method needs four ingredients: preservation losses, so that untrained words do not regress; loss weighting on the target window; varied sentences per word, so that readings generalize beyond memorized sentences; and checkpoint averaging, to limit drift. With them, the released model improves held-out literary readings by 5 pt and JKYB-Parakeet by 1.7 pt without degrading general speech.

Acknowledgments

We thank the authors of Irodori-TTS, Echo-TTS, JKYB and its Parakeet edition, kana-whisper, the JSUT and JVS corpora, Aozora Bunko, Wikipedia, the EDRDG dictionary files, JmdictFurigana, UniDic, Sudachi and Qwen.

References

- [1] C. Arata. Irodori-TTS: A flow matching-based text-to-speech model with emoji-driven style control. Hugging Face model Aratako/Irodori-TTS-v4.1-Small and <https://github.com/Aratako/Irodori-TTS>, 2026.
- [2] J. Darefsky. Echo-TTS. Blog post, <https://jordandarefsky.com/blog/2025/echo/>, 2025.
- [3] X. Liu, C. Gong, Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *ICLR*, 2023.

- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow matching for generative modeling. *ICLR*, 2023.
- [5] J. Ho, T. Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.
- [6] Meta AI. DACVAE. <https://github.com/facebookresearch/dacvae>.
- [7] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, K. Kumar. High-fidelity audio compression with improved RVQGAN. *NeurIPS*, 2023.
- [8] B. Warner et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. arXiv:2412.13663, 2024.
- [9] SB Intuitions. sbintuitions/modernbert-ja-310m. Hugging Face, 2025.
- [10] SB Intuitions. Joyo Kanji Yomi Benchmark. <https://github.com/sbintuitions/Joyo-Kanji-Yomi-Benchmark>.
- [11] Parakeet Inc. Joyo Kanji Yomi Benchmark, Parakeet Edition (JKYB-Parakeet). <https://github.com/Parakeet-Inc/Joyo-Kanji-Yomi-Benchmark-Parakeet-Edition>.
- [12] SB Intuitions. sbintuitions/kana-whisper. Hugging Face.
- [13] A. Radford et al. Robust speech recognition via large-scale weak supervision. *ICML*, 2023.
- [14] R. Sonobe, S. Takamichi, H. Saruwatari. JSUT corpus: free large-scale Japanese speech corpus for end-to-end speech synthesis. arXiv:1711.00354, 2017.
- [15] S. Takamichi et al. JVS corpus: free Japanese multi-speaker voice corpus. arXiv:1908.06248, 2019.
- [16] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [17] T. Kudo, K. Yamamoto, Y. Matsumoto. Applying conditional random fields to Japanese morphological analysis. *EMNLP*, 2004.
- [18] Y. Den, J. Nakamura, T. Ogiso, H. Ogura. A proper approach to Japanese morphological analysis: Dictionary, model, and evaluation. *LREC*, 2008.
- [19] K. Takaoka et al. Sudachi: a Japanese tokenizer for business. *LREC*, 2018.
- [20] J. Breen. Jmdict: a Japanese-multilingual dictionary. *COLING Workshop on Multilingual Linguistic Resources*, 2004.
- [21] Doublevil. JmdictFurigana. <https://github.com/Doublevil/JmdictFurigana>.
- [22] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025.
- [23] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.
- [24] M. Wortsman et al. Robust fine-tuning of zero-shot models. *CVPR*, 2022.
- [25] M. Wortsman et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *ICML*, 2022.
- [26] K. Park, S. Lee. g2pM: A neural grapheme-to-phoneme conversion package for Mandarin Chinese based on a new open benchmark dataset. *Interspeech*, 2020.